



N3 Programação Server-Side

Matheus Eduardo Pedrelli Mauricio

César Micheluzzi

Pablo Mikolaiczki

Sistema Integrado de gestão bibliotecária

Tecnologias utilizadas

Linguagem de programação:

`Python` foi a linguagem de programação utilizada para construção da api.

Ela possui diversas vantagens para construções de web apps, como uma especificação robusta entre aplicativos e servidores web, que nos proporciona frameworks web rápidos e minimalistas.

Framework Web:

`Flask` foi o framework web escolhido para o projeto, além de ser a tecnologia utilizada em sala de aula durante o semestre, é um framework minimalista que permite rapidamente criar apps funcionais.

Banco de dados:

`mysql` e `Sqlite3` foram utilizados para o desenvolvimento do projeto, sendo que, `mysql` faz parte dos requisitos do pac, e `sqlite3` é a melhor forma de realizar testes rápidos.

Acesso ao banco de dados:

`Sqlalqhemy` foi a melhor escolha para fazer acesso ao banco de dados, ele nos fornece facilidade de conexão com qualquer banco de dados relacional apenas alterando a URI de conexão. Também fornece um poderoso ORM com `Base Declarativa`, permitindo que apenas solicitamos o que deve ser feito, e não como fazer. Desta forma, dizemos ao `sqlalqhemy` que uma entidade possui relacionamentos com outra entidade, e ele se encarrega de fazer as queries complexas para retornar os dados.

Servidor web:

`gunicorn` foi o servidor web escolhido para receber as requisições, é o servidor web mais

utilizado para web apps que seguem a especificação `WSGI`

Padrão de codificação

O padrão de codificação escolhido foi o `Padrão em camadas`, onde cada camada tem a sua responsabilidade.

Camada de apresentação:

A camada de apresentação são as rotas do sistema, elas mapeiam uma url à uma função do aplicativo Flask.

Camada de aplicação:

A camada de aplicação são os serviços do sistema, eles executam lógica de negócio e retornam a resposta para a camada de apresentação.

Camada de repositório:

A camada de repositório são os métodos de acesso ao banco de dados para cada recurso, a sua função é apenas acessar o banco de dados e retornar o resultado para a camada de aplicação.

Camada de entidades:

A camada de entidades são as entidades do sistema, sua função é mapear classes em tabelas do banco de dados utilizando sqlalchemy

Funcionalidades do sistema

Recursos disponíveis

Recursos de livro:

- Listar todos os livros: `GET /api/v1/livro`
- Cadastrar um livro: `POST /api/v1/livro` + `payload json`
- Atualizar um livro: `PUT /api/v1/livro/{id}` + `payload json`
- Deletar um livro: `DELETE /api/v1/livro/{id}`

Listagem de livro

O sistema oferece para o usuário todos os livros cadastrados

Teste válido:

- Listar todos os livros: `GET /api/v1/livro`

Resposta:

`HTTP/1.1 200 OK`

```
{
  "data": [
    {
      "ISBN10": "1234567890",
      "ISBN13": "123-1234567890",
      "ano_publicacao": 2020,
      "autores": [
        {
          "id": 2,
          "nome": "Autor 2"
        },
        {
          "id": 1,
          "nome": "Autor 1"
        }
      ],
      "categoria": "Ficção",
      "created_at": "2024-07-02T20:59:34",
      "edicao": 1,
      "editora": {
        "id": 1,
        "nome": "Editora Exemplo"
      },
      "exemplares": [
        {
          "id": 431,
          "situacao": "emprestado"
        },
        {
          "id": 53534,
          "situacao": "disponivel"
        },
        {
          "id": 404932,
          "situacao": "disponivel"
        },
        {
          "id": 940530,
          "situacao": "disponivel"
        }
      ]
    }
  ]
}
```

```
        }
    ],
    "id": 1,
    "quantidade_exemplares": 4,
    "titulo": "Livro 1",
    "updated_at": "2024-07-02T20:59:34"
  }
]
}
```

Cadastro de livro

O sistema oferece para o usuário a possibilidade de em apenas uma etapa cadastrar um livro. Com apenas uma requisição com todas as informações necessárias, o sistema executa diversas lógicas de negócio. Facilitando para o usuário, porém adicionando complexidade no sistema.

Teste válido:

- Cadastro de livro: `POST /api/v1/livro`
- Payload:

```
{
  "codigo_exemplar": "5",
  "titulo": "programação server side",
  "edicao": "2",
  "autores": ["matheus", "mauricio"],
  "categoria": "eng software",
  "editora": "exemplo"
}
```

Resposta:

`HTTP/1.1 201 CREATED`

```
{
  "data": {
    "ISBN10": null,
    "ISBN13": null,
    "ano_publicacao": 2015,
    "autores": [
      {
        "id": 4,
        "nome": "matheus"
      },
      {
        "id": 5,
        "nome": "mauricio"
      }
    ],
    "categoria": "eng software",
    "created_at": "2024-07-03T13:50:33",
    "edicao": 2,
    "editora": {
      "id": 3,
      "nome": "novatec"
    },
    "exemplares": [
      {
        "id": 12345,
        "situacao": "disponivel"
      }
    ],
    "id": 6,
    "quantidade_exemplares": 1,
    "titulo": "server side",
    "updated_at": "2024-07-03T13:50:33"
  }
}
```

Atualização de livro

O sistema oferece para o usuário a possibilidade de editar qualquer quantidade de campos do livro, desde que as propriedades enviadas no payload sejam propriedades de livro.

Teste válido:

- Atualizar livro: `PUT /api/v1/livro/6`
- Payload:

```
{  
  "titulo": "livro 2048",  
  "categoria": "cyberpunk"  
}
```

Resposta:

```
HTTP/1.1 200 OK
```

```
{
  "data": {
    "ISBN10": null,
    "ISBN13": null,
    "ano_publicacao": 2015,
    "autores": [
      {
        "id": 4,
        "nome": "matheus"
      },
      {
        "id": 5,
        "nome": "mauricio"
      }
    ],
    "categoria": "cyberpunk",
    "created_at": "2024-07-03T13:50:33",
    "edicao": 2,
    "editora": {
      "id": 3,
      "nome": "novatec"
    },
    "exemplares": [
      {
        "id": 12345,
        "situacao": "disponivel"
      }
    ],
    "id": 6,
    "quantidade_exemplares": 1,
    "titulo": "livro 2048",
    "updated_at": "2024-07-03T14:03:27"
  }
}
```

Teste inválido:

- Atualizar livro: `PUT /api/v1/livro/6`
- Payload:

```
{
  "titulo": "livro 2048",
  "categoria": "cyberpunk",
  "engenharia": "software",
  "universidade": "catolica"
}
```

Resposta:

HTTP/1.1 422 UNPROCESSABLE ENTITY

```
{
  "error": "[ 'engenharia', 'universidade' ] nao sao propriedades de livro"
}
```

Exclusão de livro

O sistema oferece a possibilidade de excluir um livro, de forma que, todos os exemplares são excluídos em cascata.

Teste válido:

- Excluir livro: `DELETE /api/v1/livro/6`

Resposta:

HTTP/1.1 200 OK

```
{
  "data": "Livro deletado com sucesso"
}
```

Recursos de empréstimo:

- Listar todos os empréstimos: `GET /api/v1/emprestimo`
- Criar novo empréstimo: `POST /api/v1/emprestimo` + `payload json`

Listagem de empréstimos

O sistema permite que todos os empréstimos sejam listados, estejam eles ativos ou não.

Teste válido:

```
GET /api/v1/emprestimo
```

Resposta:

```
HTTP/1.1 200 OK
```

```
{
  "data": [
    {
      "created_at": "2024-07-02T20:59:34",
      "data_devolucao": "2024-07-17T00:00:00",
      "exemplares": [
        {
          "id": 431
        }
      ],
      "funcionario": {
        "id": 2,
        "nome": "Matheus",
        "sobrenome": "Eduardo"
      },
      "id": 1,
      "status": "Em andamento",
      "updated_at": "2024-07-02T20:59:34",
      "user": {
        "id": 1,
        "nome": "Jane",
        "sobrenome": "Smith"
      }
    }
  ]
}
```

Criação de empréstimo

O sistema permite um funcionário registrar um empréstimo de um exemplar à um usuário. Ao gerar um empréstimo, automaticamente calcula a data de devolução para 15 dias a partir da data atual, caso caia no final de semana, a data é definido na próxima segunda feira. Existem diversos tratamentos de erros, evitando que um livro emprestado seja emprestado novamente antes de ser devolvido, emprestar livro que não existe, selecionar um funcionário ou usuário que não existem.

Teste válido:

exemplar disponível, funcionário existe, usuário existe.

POST /api/v1/emprestimo

- Payload:

```
{
  "exemplar_id": "12345",
  "funcionario_id": "1",
  "usuario_id": "1"
}
```

Resposta:

HTTP/1.1 201 CREATED

```
{
  "data": {
    "created_at": "2024-07-03T14:39:36",
    "data_devolucao": "2024-07-18T00:00:00",
    "exemplares": [
      {
        "id": 12345
      }
    ],
    "funcionario": {
      "id": 1,
      "nome": "John",
      "sobrenome": "Doe"
    },
    "id": 5,
    "status": "Em andamento",
    "updated_at": "2024-07-03T14:39:36",
    "user": {
      "id": 1,
      "nome": "Jane",
      "sobrenome": "Smith"
    }
  }
}
```

Testes inválidos:

POST /api/v1/emprestimo

teste 1:

exemplar indisponível, funcionário existe, usuário existe.

Resposta:

HTTP/1.1 409 CONFLICT

```
{
  "error": "Exemplar ja emprestado"
}
```

teste 2:

exemplar disponível, funcionário não existe, usuário existe.

- Payload:

```
{
  "exemplar_id": "123",
  "funcionario_id": "10",
  "usuario_id": "1"
}
```

Resposta:

```
HTTP/1.1 404 NOT FOUND
```

```
{
  "error": "Funcionario nao encontrado"
}
```

teste 3:

exemplar disponível, funcionário existe, usuário não existe.

- Payload:

```
{
  "exemplar_id": "123",
  "funcionario_id": "1",
  "usuario_id": "10"
}
```

Resposta:

```
HTTP/1.1 404 NOT FOUND
```

```
{
  "error": "Usuario nao encontrado"
}
```

Recursos de devolução:

- Registrar devolução: `POST /api/v1/devolucao` + `payload json`

Registrar devolução

O sistema permite que um funcionário receba os livros emprestados, e encerre o empréstimo. Casos de erro também são tratados, como devolver um livro já devolvido, empréstimo não existe, funcionário não existe e usuário não existe.

Testes válido:

empréstimo existe, funcionário existe

`POST /api/v1/devolucao`

- Payload:

```
{
  "emprestimo_id": "1",
  "funcionario_id": "1",
}
```

Resposta:

`HTTP/1.1 201 CREATED`

```
{
  "data": {
    "created_at": "2024-07-03T15:43:32",
    "emprestimo_id": 1,
    "funcionario_id": 1,
    "id": 1,
    "updated_at": "2024-07-03T15:43:32"
  }
}
```

Testes inválidos:

`POST /api/v1/devolucao`

teste 1:

emprestimo não existe, funcionário existe

- Payload:

```
{
  "emprestimo_id": "10",
  "funcionario_id": "1",
}
```

Resposta:

HTTP/1.1 404 NOT FOUND

```
{
  "error": "Emprestimo not found"
}
```

teste 2:

emprestimo já finalizado

- Payload:

```
{
  "emprestimo_id": "1",
  "funcionario_id": "1",
}
```

Resposta:

HTTP/1.1 409 CONFLICT

```
{
  "error": "Emprestimo ja finalizado"
}
```

teste 3:

empréstimo não finalizado, funcionário não existe

- Payload:

```
{
  "emprestimo_id": "2",
  "funcionario_id": "10",
}
```

Resposta:

HTTP/1.1 404 NOT FOUND

```
{
  "error": "Funcionario nao encontrado"
}
```

Recursos de segurança

Atualmente a api não conta com recursos de segurança, qualquer entidade tem livre acesso à todos os recursos.

Escalabilidade

Com multiplas instâncias podemos ter uma quantidade muito grande de usuários, mas não existe nenhum tratamento de `race condition`, se múltiplos usuários tentarem alterar o mesmo recurso ao mesmo tempo, irá acontecer algum resultado inesperado, pois não há nenhum recurso como `MUTEX` para travar o acesso ao recurso enquanto em uso. Então conclui-se que o sistema não escala.

REST & RESTFul

A api segue os padrões do REST para utilizar verbos(ou métodos) HTTP para semântico o

acesso aos recursos. Utilizando o apenas nome do recurso no singular para todos os recursos daquela entidade, e nome do recurso seguido de `/id` para um único recurso daquela entidade

Rotas de livro possuem `GET`, `POST`, `PUT`, `DELETE`

Rotas de empréstimo possuem `GET`, `POST`

Rota de devolução possui `GET`